

ME/EE/CS 133b Final Project : Coverage Path Planning for Snow Shoveling

Jihyun Jeon

September 12, 2026

The following report was written for the final project of ME/CS/EE 133b, Robotics (planning, navigation, and perception), taught by Professor Günter Niemeyer. The project required a deeper exploration of a concept that was taught in class and was completed within 3 weeks.

1 Introduction

A coverage path planning (CPP) algorithm, or an algorithm that can determine a path that passes over every point in the space, are relevant in cases such as vacuum cleaning or lawn mowing, where a path that covers the whole area needs to be found. For situations like sweeping or snow shoveling, however, it is not enough to just find a path that covers an area but also has a designated start and end position.

The main objective of this project is to implement a CPP algorithm that can produce a path with a desired start and goal position. The situation this algorithm is going to be used for is for shoveling snow. Thus, the algorithm was designed such that a robot with a shovel at the front of its body can start at any position in an area filled uniformly with snow and shovel all snow into a designated goal position.

2 Requirements

The requirements for such an algorithm is as follows:

1. The robot must move through all points completely such that at the end of its path, the only area with snow is the goal position.

2. The algorithm must be able to find a path for any map, start position, and goal position (i.e. is systematic) given that the map is fully connected.
3. The robot has complete knowledge of the map and can't move into any walls or obstacles.
4. The robot can turn a maximum of 90 degrees. Thus, the robot can't move in diagonals (as this will cause turns of 135 degrees) or turn backwards.
5. The algorithm should be able to find the optimal path.

While it is not possible to find an algorithm that satisfies all of the requirements (as to be illustrated in section 5), the algorithm was designed with these requirements in mind.

3 Comparison between CPP algorithms

Various CPP algorithms were compared, with the help of surveys of CPP algorithms^{1, 2}. As we are assuming complete knowledge of the map, only offline algorithms were considered.

Most CPP algorithms fall in one of three categories depending on how they treat the coverage space: graph based, cell decomposition based, or grid based. The graph based algorithms are mostly used for coverage of roads and are not suitable for the purposes of this project. Cell decomposition algorithms generally work by dividing the space into cells according to critical points, such as points of obstacles, and then traversing through each cell in a zigzag path. While the cell decomposition algorithms are good for ensuring complete coverage and dealing with irregularly shaped obstacles, most require many sharp turns and don't have as much capacity to add start and goal positions. On the contrary, it is easy to set start and goal positions in grid based algorithms and add the constraint of requirement 4 by restricting motions to up, down, left and right. Also, grids have the added convenience of keeping track of the amount of snow in each grid cell and are capable of representing irregular shapes as long as the resolution is high enough. Thus, the algorithm for this project was chosen to be grid based.

Of grid based algorithms, the Spanning Tree algorithm and Wavefront algorithm were considered. The Spanning Tree algorithm works by generating a minimum spanning tree that expands from the start position and spiraling around the edges of the tree. The Spanning Tree algorithm is simple to implement and ensures a coverage path. However, it has the constraint that the start and end position of the path is the same (i.e. it can only produce a cycle), which is not appropriate

for this project. The Wavefront algorithm works by giving each grid cell a distance cost value with wave propagation from the goal cell and creating a path from the start position by prioritizing a higher distance cost. The Wavefront algorithm was ultimately chosen for its capabilities of adding start and goal positions and changing the goal of the path with different distance cost values.

4 Wavefront algorithm and Implementation

The initial implementation of the wavefront algorithm was based off of the pseudocode found in a paper on an online wavefront algorithm³. The algorithm was implemented with Python and visualized using a modified version of the Visualization class provided with homework 5.

The first step of the wavefront algorithm is to calculate the distance cost value of each grid cell. The goal is given a value of 0 and a "wave" propagates out from the goal, with the cells on each step of the wave getting increased values. The distance cost value is essentially the number of steps it takes to get to the goal. Performing the Distance Transform has time complexity of $O(rows \times cols)$ and is done as follows:

Algorithm 1 Wavefront Distance Transform

```

1: procedure DISTANCE TRANSFORM(goal)
2:   queue  $\leftarrow$  [goal]
3:   visited  $\leftarrow$  [goal]
4:   directions  $\leftarrow$  [goal]
5:   dists  $\leftarrow$  zeros(rows, cols)
6:   while queue is not empty do
7:     row, col  $\leftarrow$  queue.pop()
8:     for drow, dcol  $\in$  [(0, 1), (1, 0), (-1, 0), (0, -1)] do
9:       nextRow  $\leftarrow$  row + drow
10:      nextCol  $\leftarrow$  col + dcol
11:      if [nextRow, nextCol] is Free and [nextRow, nextCol]  $\notin$  visited then
12:        dists[nextRow][nextCol]  $\leftarrow$  dists[row][col] + 1
13:        queue.append([nextRow, nextCol])
14:        visited.append([nextRow, nextCol])

```

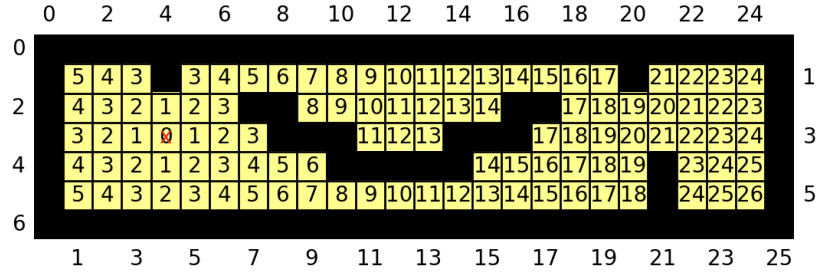


Figure 1: Example of completed distance transform for goal position of (3, 4)

Once this is done, the calculated distance cost is used to generate a coverage path from start to goal. This is done by traversing towards the neighbor that is unvisited and has the highest distance value from a cell. The wavefront algorithm has time complexity of $O(rows \times cols)$

Algorithm 2 Path Generation

```

1: procedure CPP PATH
2:   visited  $\leftarrow []$ 
3:   path  $\leftarrow [start]$ 
4:   curr  $\leftarrow start$ 
5:   while curr  $\neq$  goal do
6:     maxD  $\leftarrow -1$ 
7:     nextNode  $\leftarrow None$ 
8:     row, col  $\leftarrow curr[0], curr[1]$ 
9:     for drow, dcol  $\in [(0, 1), (1, 0), (-1, 0), (0, -1)]$  do
10:      nextRow  $\leftarrow row + drow$ 
11:      nextCol  $\leftarrow col + dcol$ 
12:      if [nextRow, nextCol] is Free and [nextRow, nextCol]  $\notin$  visited
and dists[nextRow][nextCol]  $>$  maxD then
13:        nextNode  $\leftarrow [nextRow, nextCol]$ 
14:        maxD  $\leftarrow dists[nextRow][nextCol]$ 
15:      if nextNode = None then return path
16:      path.append(nextNode)
17:      curr  $\leftarrow nextNode$ 
return path

```

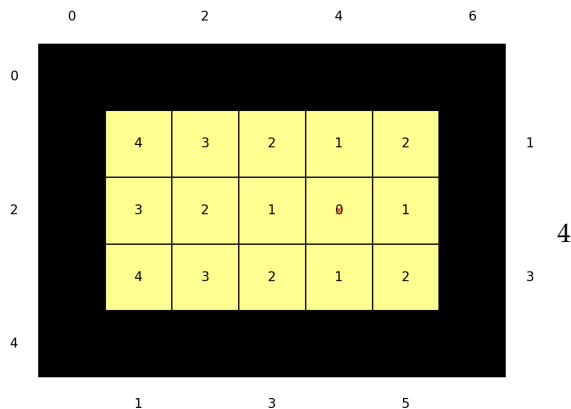


Figure 2: Example of distance map of a 5 x 3 grid with goal at (2, 4)

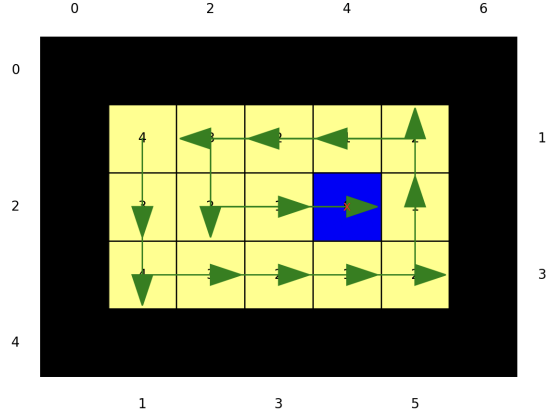


Figure 3: Example of coverage path from (1, 1) to (2, 4)

5 Reevaluation of Requirements

However, the enforcement of all of the requirements on the wavefront algorithm causes it to not work for certain cases, making the algorithm not systematic.

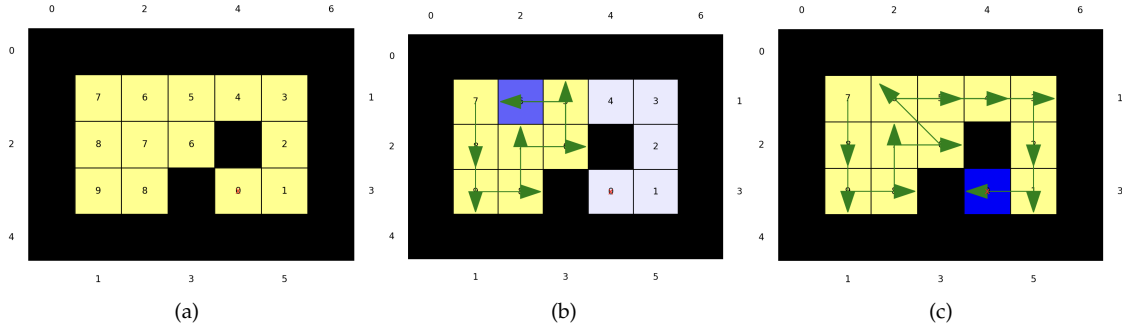


Figure 4: Failure of algorithm for map (a) with start position of (0,0) and goal position of (3, 4). This can be fixed by allowing diagonals like (c)

For the case shown in figure 4, there is no possible path from the start to goal as the algorithm causes the path to go into a dead end when traversing the cell with transformed distance of 6. This is due to requirement 4. In order to find a coverage path from start to goal, requirement 4 needs to be relieved so either backtracking has to be allowed to let the robot back out of a dead end or

diagonal movement, as shown in (c), needs to be allowed.

6 Modified Wavefront algorithm

6.1 Allowing Diagonals

The initial attempt to modify the wavefront algorithm was allowing diagonal movement. However, while this does solve the case of Figure 4, it is not a general solution to the algorithm as illustrated in the situation in Figure 5. Getting stuck in a dead end doesn't happen because the movement is restricted in direction, but rather a limitation of the wavefront algorithm and the enforcement of not being able to go to a node that's already been visited. However, the case in Figure 5 shows more promise in getting solved as there are more possible choices that can be made. For example, in Figure 5, instead of moving from (3, 21) to (3, 22), the robot could have moved to (4, 22) without violating the algorithm and wouldn't have gotten stuck.

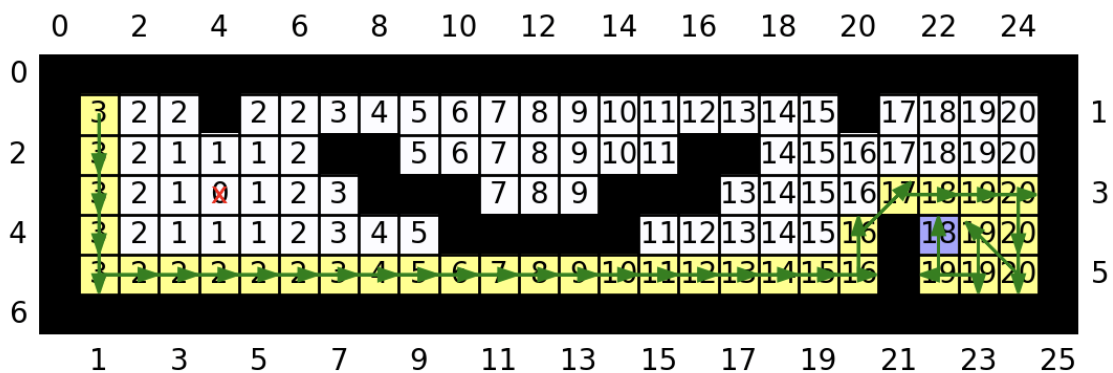


Figure 5: Even with diagonals, the algorithm gets stuck in a dead end situation

6.2 Depth First Search

Algorithm 3 Path Generation Recursion

```

1: procedure CPP RECURSIVE PATH(curr, path, visited)
2:   if curr = goal then return path
3:   maxD  $\leftarrow$  -1
4:   maxNodes  $\leftarrow$  []
5:   row, col  $\leftarrow$  curr[0], curr[1]
6:   for drow, dcol  $\in$  [(0, 1), (1, 0), (-1, 0), (0, -1)] do
7:     nextRow  $\leftarrow$  row + drow
8:     nextCol  $\leftarrow$  col + dcol
9:     if [nextRow, nextCol] is Free and [nextRow, nextCol]  $\notin$  visited then
10:      nextNode  $\leftarrow$  [nextRow, nextCol]
11:      if dists[nextRow][nextCol] = maxD then
12:        maxNodes.append(nextNode)
13:      if dists[nextRow][nextCol] > maxD then
14:        maxD  $\leftarrow$  dists[nextRow][nextCol]
15:        maxNodes  $\leftarrow$  [nextNode]
16:      visited.append(nextNode)
17:      if maxD = -1 then return None
18:      for nextNode  $\in$  maxNodes do
19:        path.append(nextNode)
20:        curr  $\leftarrow$  nextNode
21:        result  $\leftarrow$  CPP RECURSIVE PATH(curr, path, visited)
22:        if result  $\neq$  None then return result
    return None

```

The Depth First Search wavefront algorithm is computationally heavy, with a worst case time complexity of $O(e^{rows \times cols})$.

Additionally, this is not a solution for making the algorithm systematic as there are start and goal configurations where no coverage path exists when backtracking is banned. For the map of Figure 6, the narrow pathways at row 1 and 5 means that once the robot passes the pathway, it is unable to return. This means it is impossible for the robot to completely cover the area. For example, with the uncovered (4, 15 - 19) cells, it is impossible for the robot to cover them if it approaches them from the left as the wavefront algorithm pushes the robot towards higher distance values before they are completely covered. The only way they will be covered is if the robot approached them from the right, meaning the robot would've passed through row 1 from left to right. However, the same situation happens to cells (2, 9 - 15) and (3, 11 - 13) as the wavefront

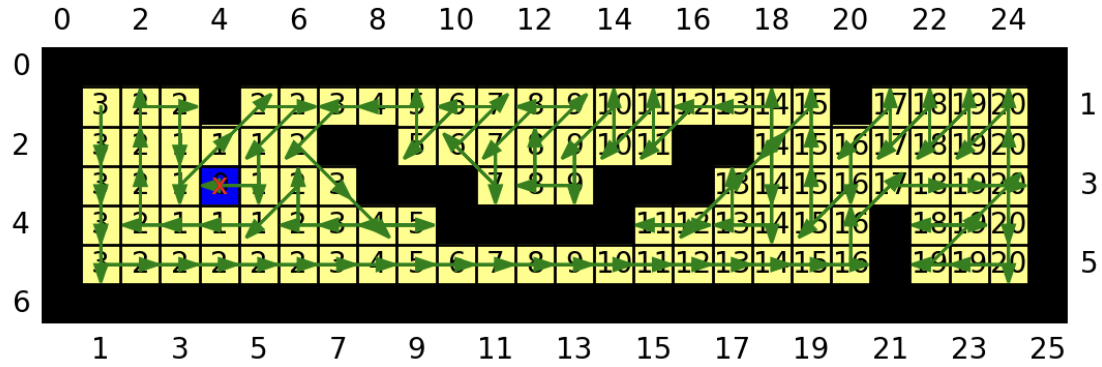


Figure 7: Example of successful coverage path finding with backtracking enabled

Algorithm 4 Path Generation with Back Tracking

```

1: procedure CPP PATH BACKTRACKING
2:   visited  $\leftarrow []$ 
3:   path  $\leftarrow [start]$ 
4:   curr  $\leftarrow start$ 
5:   while curr  $\neq goal$  and the map isn't fully covered do
6:     maxD  $\leftarrow -1$ 
7:     maxIdx  $\leftarrow -1$ 
8:     nextNode  $\leftarrow None$ 
9:     for  $i \in range(len(path))$  do
10:      curr  $\leftarrow path[len(path) - 1 - i]$ 
11:      row, col  $\leftarrow curr[0], curr[1]$ 
12:      for  $drow, dcol \in [(0, 1), (1, 0), (-1, 0), (0, -1)]$  do
13:        nextRow  $\leftarrow row + drow$ 
14:        nextCol  $\leftarrow col + dcol$ 
15:        if [nextRow, nextCol] is Free and [nextRow, nextCol]  $\notin visited$ 
and dists[nextRow][nextCol]  $> maxD$  then
16:          nextNode  $\leftarrow [nextRow, nextCol]$ 
17:          maxD  $\leftarrow dists[nextRow][nextCol]$ 
18:          maxIdx  $\leftarrow i$ 
19:        if maxD  $> 0$  then Break
20:      if nextNode = None then return path
21:      if maxIdx  $> 0$  then
22:        Add path backtracking to path[len(path) - 1 - maxIdx] to path
23:        path.append(nextNode)
24:        curr  $\leftarrow nextNode$ 
return path

```

Backtracking ensures Requirements 1, 2, 3 are always satisfied (i.e. it is always able to find a complete coverage path for any map, start position and goal position) regardless of whether or not diagonals are allowed. With backtracking, the wavefront algorithm is able to work in its full potential and utilizes the Distance Transform values to make the robot cover the map.

Backtracking adds additional time complexity to the original wavefront algorithm, with a worst case time complexity of $O((rows \times cols)^2)$. However, compared to the DFS algorithm that has exponential time complexity, the backtracking algorithm is much better.

7 Optimized Modified Wavefront algorithm

7.1 Improving Backtracking

Initially, the path backtracking from $path[len(path) - 1 - maxIdx]$ to $path$ was found by traversing backwards through the existing path until the desired $path[len(path) - 1 - maxIdx]$ was found. To make sure this doesn't interfere with the CPP algorithm, the backtracking that needed to happen at a given step of a path was taken track of and the actual backtracking was added to the path in post processing. While this worked, this meant the robot needed to follow the path backwards to backtrack to any given node regardless of the distance to the node, creating a not optimal path.

To improve backtracking and minimize the length of the path, Dijkstra's algorithm was implemented to find the shortest backtracking path. Similarly, to make sure the backtracking doesn't interfere with the CPP algorithm, the backtracking path was added in post processing. Also, to make sure the CPP algorithm could properly keep track of visited nodes, the Dijkstra's algorithm was only used to find a path through already explored nodes.

Implementing Dijkstra's algorithm was very effective in optimizing the path. For the map shown in Figure 7, the path was initially 1004 steps without optimizing and is 105 steps with Dijkstra's algorithm.

7.2 Improving Exploration

A possible consideration in the wavefront algorithm is the order of searching at a given node. The backtracking algorithm doesn't explore all possible paths like the dfs algorithm does and the way the next node to explore is chosen affects the final path. Various permutations of directions were

explored for the map of Figure 7, with the maximum path with Dijkstra's algorithm requiring 107 steps and the minimum path with Dijkstra's algorithm requiring 105 steps. While the number of backtracking required was the same for both paths, the length of the path required to backtrack to the next node was different which affected the final length of the paths.

While it is possible to consider all permutations of possible search orders to find the shortest path, there are 40320 total possible permutations when diagonal movements are allowed, making it computationally heavy. One possible way of optimizing exploration is changing the order of search based on the start and goal positions such that movement away from the goal position is prioritized.

8 Lessons Learned and Conclusion

This project was an in depth exploration of the wavefront algorithm when constraints as illustrated in section 2 are enforced. While the reference papers implied that it is always possible to find a coverage path with a set start and goal position, that was not the case. Mainly two methods were attempted to solve this problem, a depth first search and backtracking, and backtracking was successful in creating a systematic coverage path planning algorithm.

In the process of doing this project, several ideas for future work occurred. First, while spanning tree algorithm was initially dismissed, ideas of utilizing them to meet the requirements occurred. It might be possible to modify the tree to incorporate start and goal positions, especially as the Distance Transform is essentially building a spanning tree that has the goal as the root.

Another possible idea is to think of the snow plow problem, where when the robot passes by a spot it doesn't push the snow forward but rather pushes it to the side of the robot. Modifications to the way the Distance Transformation is done could be used to create an algorithm that pushes all snow into a single goal position.

9 Works Referenced

1. Galceran, Enric, and Marc Carreras. "A survey on coverage path planning for robotics." *Robotics and Autonomous Systems*, vol. 61, no. 12, 2013, pp. 1258-1276. ScienceDirect, <https://www.sciencedirect.com/science/article/pii/S092188901300167X#>

s000005.

2. Fevgas, Georgio, et al. "Coverage Path Planning Methods Focusing on Energy Efficient and Cooperative Strategies for Unmanned Aerial Vehicles." *Wireless Sensing and Networking for the Internet of Things*, vol. 22, no. 3, 2022, p. NA. MDPI, <https://www.mdpi.com/1424-8220/22/3/1235>.
3. Zelinsky, Alexander, et al. *Planning Paths of Complete Coverage of an Unstructured Environment by a Mobile Robot*. Dec 2006.